

# The Khronos OpenGL API Registry Schema

Jon Leech

Last updated 2020-10-27

## Abstract

This document describes the Khronos OpenGL API Registry XML schema, and provides some additional information about using the registry and scripts to generate C header files. The underlying XML files and scripts, as well as headers generated from the current authoritative copy of the XML files, can be obtained from the OpenGL-Registry github repository at

<https://github.com/KhronosGroup/OpenGL-Registry>

## Contents

|          |                                                                 |          |
|----------|-----------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                             | <b>3</b> |
| <b>2</b> | <b>Getting Started</b>                                          | <b>3</b> |
| 2.1      | Header Generation Script - <code>genheaders.py</code> . . . . . | 4        |
| 2.2      | Registry Processing Script - <code>reg.py</code> . . . . .      | 4        |
| <b>3</b> | <b>XML Registry Schema</b>                                      | <b>4</b> |
| 3.1      | Profiles . . . . .                                              | 5        |
| 3.2      | API Names . . . . .                                             | 5        |
| <b>4</b> | <b>Registry Root (&lt;registry&gt; tag)</b>                     | <b>5</b> |
| 4.1      | Attributes of <registry> tags . . . . .                         | 5        |
| 4.2      | Contents of <registry> tags . . . . .                           | 5        |
| <b>5</b> | <b>API types (&lt;types&gt; tag)</b>                            | <b>6</b> |
| 5.1      | Attributes of <type> tags . . . . .                             | 6        |
| 5.2      | Contents of <type> tags . . . . .                               | 6        |
| 5.3      | Example of a <types> tag . . . . .                              | 6        |
| <b>6</b> | <b>Enumerant Groups (&lt;groups&gt; tag)</b>                    | <b>7</b> |
| 6.1      | Attributes of <groups> tags . . . . .                           | 7        |
| 6.2      | Contents of <groups> tags . . . . .                             | 7        |
| 6.3      | Example of <groups> tags . . . . .                              | 7        |

|           |                                                      |           |
|-----------|------------------------------------------------------|-----------|
| <b>7</b>  | <b>Enumerant Group (&lt;group&gt; tag)</b>           | <b>7</b>  |
| 7.1       | Attributes of <group> tags . . . . .                 | 7         |
| 7.2       | Contents of <group> tags . . . . .                   | 8         |
| 7.3       | Meaning of <group> tags . . . . .                    | 8         |
| <b>8</b>  | <b>Enumerant Blocks (&lt;enums&gt; tag)</b>          | <b>8</b>  |
| 8.1       | Attributes of <enums> tags . . . . .                 | 8         |
| 8.2       | Contents of <enums> tags . . . . .                   | 8         |
| 8.3       | Example of <enums> tags . . . . .                    | 9         |
| <b>9</b>  | <b>Enumerants (&lt;enum&gt; tag)</b>                 | <b>9</b>  |
| 9.1       | Attributes of <enum> tags . . . . .                  | 9         |
| 9.2       | Contents of <enum> tags . . . . .                    | 10        |
| <b>10</b> | <b>Unused Enumerants (&lt;unused&gt; tag)</b>        | <b>10</b> |
| 10.1      | Attributes of <unused> tags . . . . .                | 10        |
| 10.2      | Contents of <unused> tags . . . . .                  | 10        |
| <b>11</b> | <b>Command Blocks (&lt;commands&gt; tag)</b>         | <b>10</b> |
| 11.1      | Attributes of <commands> tags . . . . .              | 10        |
| 11.2      | Contents of <commands> tags . . . . .                | 10        |
| <b>12</b> | <b>Commands (&lt;command&gt; tag)</b>                | <b>11</b> |
| 12.1      | Attributes of <command> tags . . . . .               | 11        |
| 12.2      | Contents of <command> tags . . . . .                 | 11        |
| 12.3      | Command prototype (<proto> tags) . . . . .           | 11        |
| 12.3.1    | Attributes of <proto> tags . . . . .                 | 11        |
| 12.3.2    | Contents of <proto> tags . . . . .                   | 12        |
| 12.4      | Command parameter (<param> tags) . . . . .           | 12        |
| 12.4.1    | Attributes of <param> tags . . . . .                 | 12        |
| 12.4.2    | Contents of <param> tags . . . . .                   | 12        |
| 12.5      | Example of a <commands> tag . . . . .                | 13        |
| <b>13</b> | <b>API Features / Versions (&lt;feature&gt; tag)</b> | <b>13</b> |
| 13.1      | Attributes of <feature> tags . . . . .               | 13        |
| 13.2      | Contents of <feature> tags . . . . .                 | 13        |
| 13.3      | Example of a <feature> tag . . . . .                 | 14        |
| <b>14</b> | <b>Extension Blocks (&lt;extensions&gt; tag)</b>     | <b>14</b> |
| 14.1      | Attributes of <extensions> tags . . . . .            | 14        |
| 14.2      | Contents of <extensions> tags . . . . .              | 14        |
| <b>15</b> | <b>API Extensions (&lt;extension&gt; tag)</b>        | <b>15</b> |
| 15.1      | Attributes of <extension> tags . . . . .             | 15        |
| 15.2      | Contents of <extension> tags . . . . .               | 15        |
| 15.3      | Example of an <extensions> tag . . . . .             | 15        |

|                                                                                     |           |
|-------------------------------------------------------------------------------------|-----------|
| <b>16 Required and Removed Interfaces (&lt;require&gt; and &lt;remove&gt; tags)</b> | <b>16</b> |
| 16.1 Attributes of <require> and <remove> tags . . . . .                            | 16        |
| 16.2 Contents of <require> and <remove> tags . . . . .                              | 16        |
| <b>17 General Discussion</b>                                                        | <b>17</b> |
| 17.1 Stability of the XML Database and Schema . . . . .                             | 17        |
| 17.2 Feature Enhancements to the Registry . . . . .                                 | 17        |
| 17.3 Type Annotations and Relationship to .spec Files . . . . .                     | 17        |
| 17.3.1 Simple API Type Aliases . . . . .                                            | 18        |
| 17.3.2 Numeric Constraints . . . . .                                                | 18        |
| 17.3.3 GL Object Names . . . . .                                                    | 18        |
| 17.3.4 Groups Not Defined Yet . . . . .                                             | 18        |
| 17.3.5 Other Groups . . . . .                                                       | 19        |
| 17.3.6 Validating Groups . . . . .                                                  | 19        |
| <b>18 Change Log</b>                                                                | <b>19</b> |

## 1 Introduction

The registry uses an XML representation of the API and a set of Python 3 scripts to manipulate the XML, based on the lxml Python bindings. It comes with an XML schema and validator. Outside users are welcome to submit pull requests on github, although the XML schema itself is mature and unlikely to change at this point.

Other APIs defined by Khronos, such as Vulkan and OpenXR, use a similar XML schema originally based on the OpenGL schema to describe those APIs.

Some groups outside Khronos have their own XML based API descriptions, often used for additional purposes such as library code generators or extension loaders, and it may be desirable to construct XSLT or other translators between the schema.

## 2 Getting Started

Once the repository has been downloaded, if you're running in a Linux command-line environment and have Python 3, the lxml Python bindings, and libxml installed, you should just be able to invoke `make` and generate C/C++ header files for all the following targets:

- `GL/glext.h` - OpenGL 1.2 (and later) compatibility profile API + extensions
- `GL/glcorearb.h` - OpenGL core profile API + extensions
- `GLLES/gl.h` - OpenGL compatibility profile API
- `GLLES/glext.h` - OpenGL ES 1.x extensions
- `GLLES2/gl2.h` - OpenGL ES 2.x API
- `GLLES2/gl2ext.h` - OpenGL ES 2.x extensions

- `GLLES3/gl3.h` - OpenGL ES 3.x API
- `GL/glx.h` - GLX API
- `GL/glxext.h` - GLX 1.3 (and later) API + extensions
- `GL/wgl.h` - WGL API
- `GL/wglxext.h` - WGL extensions
- `EGL/egl.h` - EGL (still being worked on)

Starting with the Makefile rules and inspecting the files `gl.xml`, `genheaders.py`, and `reg.py` will be necessary if you want to repurpose the registry for reasons other than header file generation, or to generate headers for languages other than C.

If you're running in a Microsoft Windows, MacOS X, or other non-Linux environment, there are equivalent versions of Python and GNU Make, although we haven't tested this ourselves. Feedback would be helpful. If you're using Windows 10, you should probably try to build in the Ubuntu Linux subsystem rather than older Unix emulation environments such as Cygwin.

## 2.1 Header Generation Script - `genheaders.py`

When generating header files using the `genheaders.py` script, an API name and profile name are required, as shown in the Makefile examples. Additionally, specific versions and extensions can be required or excluded. Based on this information, the generator script extracts the relevant interfaces and creates a C-language header file for them. `genheaders.py` contains predefined generator options for OpenGL compatibility and core profiles, OpenGL ES 1 / 2 / 3, GLX, and WGL headers.

The generator script is intended to be generalizable to other languages by writing new generator classes. Such generators would have to rewrite the C types and definitions in the XML to something appropriate to their language.

## 2.2 Registry Processing Script - `reg.py`

Actual XML registry processing is done in `reg.py`, which contains several objects and methods for loading registries and extracting interfaces and extensions for use in header generation. There is some internal documentation in the form of comments although nothing more extensive exists yet, and it's possible the Python scripts will evolve significantly based on public feedback.

# 3 XML Registry Schema

The format of an XML registry is a top level `<registry>` tag containing `<types>`, `<enums>`, `<commands>`, `<feature>`, and `<extension>` tags describing the different elements of an API, as explained below. This description corresponds to a formal

Relax NG schema file, `registry.rnc`, against which the XML registry files can be validated.

At present there are separate registries for:

- OpenGL and OpenGL ES - `gl.xml`
- GLX - `glx.xml`
- WGL - `wgl.xml`
- EGL - `egl.xml` (still in development)

### 3.1 Profiles

Types and enumerants can have different definitions depending on the API profile requested, which allows us to accomodate minor incompatibilities in the OpenGL and OpenGL ES APIs, for example. Features and extensions can include some elements conditionally depending on the API profile requested.

### 3.2 API Names

Several tags use a `api` attribute. This is an arbitrary string, specified at header generation time, for labelling properties of a specific API. The string can be, but is not necessarily, an actual API name. As used in `genheaders.py` and `gl.xml`, the API names are `gl`, `gles1`, and `gles2`, corresponding to OpenGL, OpenGL ES 1, and OpenGL ES 2/3, respectively.

## 4 Registry Root (`<registry>` tag)

A `<registry>` contains the entire definition of one or more related APIs.

### 4.1 Attributes of `<registry>` tags

None.

### 4.2 Contents of `<registry>` tags

Zero or more of each of the following tags, normally in this order (although order shouldn't be important):

- `<comment>` - Contains arbitrary text, such as a copyright statement. Unused.
- `<types>` (see section 5) - defines API types. Usually only one tag is used.
- `<groups>` (see section 6) - defines named groups of tokens for possible parameter validation in API bindings for languages other than C. Usually only one tag is used.

- `<enums>` (see section 8) - defines API enumerants (tokens). Usually multiple tags are used.
- `<commands>` (see section 11) - defines API commands (functions). Usually only one tag is used.
- `<feature>` (see section 13) - defines API feature interfaces (API versions, more or less). One tag per feature set.
- `<extensions>` (see section 14) - defines API extension interfaces. Usually only one tag is used, wrapping many extensions.

## 5 API types (`<types>` tag)

The `<types>` tag contains individual `<type>` tags describing each of the derived types used in the API.

Each `<type>` tag contains legal C code, with attributes or embedded tags denoting the type name.

### 5.1 Attributes of `<type>` tags

- `requires` - another type name this type requires to complete its definition.
- `name` - name of this type (if not defined in the tag body).
- `api` - an API name (see `<feature>` below) which specializes this definition of the named type, so that the same API types may have different definitions for e.g. GL ES and GL.
- `comment` - arbitrary string (unused).

### 5.2 Contents of `<type>` tags

`<type>` contains text which is legal C code for a type declaration. It may also contain embedded tags:

- `<apientry/>` - insert a platform calling convention macro here during header generation, used mostly for function pointer types.
- `<name>` - contains the name of this type (if not defined in the tag attributes).

### 5.3 Example of a `<types>` tag

```
<types>
  <type name="stddef"><![CDATA[#include <stddef.h>]]></type>
  <type requires="stddef">typedef ptrdiff_t <name>GLintptr</name>;</type>
</types>
```

If the `GLint64` type is required by a command, this will result in the following declarations:

```
#include <stddef.h>
typedef ptrdiff_t GLintptr;
```

## 6 Enumerant Groups (<groups> tag)

The <groups> tags contain individual <group> tags describing some of the group annotations used for return and parameter types.

### 6.1 Attributes of <groups> tags

None.

### 6.2 Contents of <groups> tags

Each <groups> block contains zero or more <group> tags, in arbitrary order (although they are typically ordered by group name, to improve human readability).

### 6.3 Example of <groups> tags

```
<groups>
  <group name="AccumOp">
    <enum name="GL_ACCUM"/>
  </group>

  <group name="AttribMask">
    <enum name="GL_ACCUM_BUFFER_BIT"/>
    <enum name="GL_ALL_ATTRIB_BITS"/>
  </group>
</groups>
```

## 7 Enumerant Group (<group> tag)

Each <group> tag defines a single group annotation.

### 7.1 Attributes of <group> tags

- name - group name, an arbitrary string for grouping a set of enums together within a broader namespace.

## 7.2 Contents of `<group>` tags

`<group>` tags may contain zero or more `<enum>` tags. Each `<enum>` tag may contain only a `name` attribute, which should correspond to a `<enum>` definition in an `<enums>` block.

## 7.3 Meaning of `<group>` tags

If a `<proto>` or `<param>` tag of a `<command>` has a `group` attribute defined, and that attribute matches a `<group>` name, then the return type or parameter type is considered to be constrained to values defined by the corresponding `<group>`. C language bindings do not attempt to enforce this constraint in any way, but other language bindings may try to do so.

# 8 Enumerant Blocks (`<enums>` tag)

The `<enums>` tags contain individual `<enum>` tags describing each of the token (enumerant) names used in the API.

## 8.1 Attributes of `<enums>` tags

- `namespace` - a string for grouping many different enums together, currently unused but typically something like `GL` for all enums in the OpenGL / OpenGL ES shared namespace. Multiple `<enums>` tags can share the same namespace.
- `type` - a string describing the data type of the values of this group of enums, currently unused. The only string used at present in the is `bitmask`.
- `start, end` - integers defining the start and end of a reserved range of enumerants for a particular vendor or purpose. `start` must be  $\leq$  `end`. These fields define formal enumerant allocations within a namespace, and are made by the Khronos Registrar on request from implementers following the enum allocation policy.
- `vendor` - string describing the vendor or purposes to whom a reserved range of enumerants is allocated.
- `comment` - arbitrary string (unused)

## 8.2 Contents of `<enums>` tags

Each `<enums>` block contains zero or more `<enum>` and `<unused>` tags, in arbitrary order (although they are typically ordered by sorting on enumerant values, to improve human readability).



### 8.3 Example of <enums> tags

```
<enums namespace="AttribMask" type="bitmask">
  <enum value="0x00000001" name="GL_CURRENT_BIT" />
  <enum value="0x00000002" name="GL_POINT_BIT" />
</enums>
<enums namespace="GL" start="0x80E0" end="0x810F" vendor="MS">
  <enum value="0x80E0" name="GL_BGR" />
  <unused start="0x80E1" end="0x810F" />
</enums>
```

When processed into a C header, and assuming all these tokens were required, this results in

```
#define GL_CURRENT_BIT 0x00000001
#define GL_POINT_BIT 0x00000001
#define GL_BGR 0x80E0
```

## 9 Enumerants (<enum> tag)

Each <enum> tag defines a single GL (or other API) token.

### 9.1 Attributes of <enum> tags

- **value** - enumerant value, a legal C constant (usually a hexadecimal integer).
- **name** - enumerant name, a legal C preprocessor token name.
- **api** - an API name which specializes this definition of the named enum, so that different APIs may have different values for the same token (used to address a few accidental incompatibilities between GL and GL ES).
- **type** - legal C suffix for the value to force it to a specific type. Currently only `u` and `ull` are used, for unsigned 32- and 64-bit integer values, respectively. Separated from the `value` field since this eases parsing and sorting of values, and is rarely used.
- **group** - group name, an arbitrary string for grouping a set of enums together within a broader namespace.

**Note:** the <group> tag (see section 7) serves the same purpose, but is deprecated in favor of this attribute. Eventually all <group> tags will be removed from the XML registries.

- **alias** - name of another enumerant this is an alias of, used where token names have been changed as a result of profile changes or for consistency purposes. An enumerant alias is simply a different name for the exact same value. At present, enumerants which are promoted from extension to core API status are

## 7.2 Contents of `<group>` tags

`<group>` tags may contain zero or more `<enum>` tags. Each `<enum>` tag may contain only a `name` attribute, which should correspond to a `<enum>` definition in an `<enums>` block.

## 7.3 Meaning of `<group>` tags

If a `<proto>` or `<param>` tag of a `<command>` has a `group` attribute defined, and that attribute matches a `<group>` name, then the return type or parameter type is considered to be constrained to values defined by the corresponding `<group>`. C language bindings do not attempt to enforce this constraint in any way, but other language bindings may try to do so.

# 8 Enumerant Blocks (`<enums>` tag)

The `<enums>` tags contain individual `<enum>` tags describing each of the token (enumerant) names used in the API.

## 8.1 Attributes of `<enums>` tags

- `namespace` - a string for grouping many different enums together, currently unused but typically something like `GL` for all enums in the OpenGL / OpenGL ES shared namespace. Multiple `<enums>` tags can share the same namespace.
- `type` - a string describing the data type of the values of this group of enums, currently unused. The only string used at present in the is `bitmask`.
- `start, end` - integers defining the start and end of a reserved range of enumerants for a particular vendor or purpose. `start` must be  $\leq$  `end`. These fields define formal enumerant allocations within a namespace, and are made by the Khronos Registrar on request from implementers following the enum allocation policy.
- `vendor` - string describing the vendor or purposes to whom a reserved range of enumerants is allocated.
- `comment` - arbitrary string (unused)

## 8.2 Contents of `<enums>` tags

Each `<enums>` block contains zero or more `<enum>` and `<unused>` tags, in arbitrary order (although they are typically ordered by sorting on enumerant values, to improve human readability).

### 8.3 Example of <enums> tags

```
<enums namespace="AttribMask" type="bitmask">
  <enum value="0x00000001" name="GL_CURRENT_BIT" />
  <enum value="0x00000002" name="GL_POINT_BIT" />
</enums>
<enums namespace="GL" start="0x80E0" end="0x810F" vendor="MS">
  <enum value="0x80E0" name="GL_BGR" />
  <unused start="0x80E1" end="0x810F" />
</enums>
```

When processed into a C header, and assuming all these tokens were required, this results in

```
#define GL_CURRENT_BIT 0x00000001
#define GL_POINT_BIT 0x00000001
#define GL_BGR 0x80E0
```

## 9 Enumerants (<enum> tag)

Each <enum> tag defines a single GL (or other API) token.

### 9.1 Attributes of <enum> tags

- **value** - enumerant value, a legal C constant (usually a hexadecimal integer).
- **name** - enumerant name, a legal C preprocessor token name.
- **api** - an API name which specializes this definition of the named enum, so that different APIs may have different values for the same token (used to address a few accidental incompatibilities between GL and GL ES).
- **type** - legal C suffix for the value to force it to a specific type. Currently only `u` and `ull` are used, for unsigned 32- and 64-bit integer values, respectively. Separated from the `value` field since this eases parsing and sorting of values, and is rarely used.
- **group** - group name, an arbitrary string for grouping a set of enums together within a broader namespace.

**Note:** the <group> tag (see section 7) serves the same purpose, but is deprecated in favor of this attribute. Eventually all <group> tags will be removed from the XML registries.

- **alias** - name of another enumerant this is an alias of, used where token names have been changed as a result of profile changes or for consistency purposes. An enumerant alias is simply a different name for the exact same value. At present, enumerants which are promoted from extension to core API status are



## 12.5 Example of a `<commands>` tag

```
<commands>
  <command>
    <proto>void <name>glBeginConditionalRenderNV</name></proto>
    <param><ptype>GLuint</ptype> <name>id</name></param>
    <param><ptype>GLenum</ptype> <name>mode</name></param>
    <alias name="glBeginConditionalRender" />
    <glx type="render" opcode="348" />
  </command>
</commands>
```

When processed into a C header, this results in

```
void glBeginConditionalRenderNV(GLuint id, GLenum mode);
```

## 13 API Features / Versions (`<feature>` tag)

API features are described in individual `<feature>` tags. A feature is the set of interfaces (enumerants and commands) defined by a particular API and version, such as OpenGL 4.0 or OpenGL ES 3.0, and includes all API profiles of that version.

### 13.1 Attributes of `<feature>` tags

- `api` - API name this feature is for (see section 3.2), such as `gl` or `gles2`.
- `name` - version name, used as the C preprocessor token under which the version's interfaces are protected against multiple inclusion. Example: `GL_VERSION_4_2`.
- `protect` - an additional preprocessor token used to protect a feature definition. Usually another feature or extension `name`. Rarely used, for odd circumstances where the definition of a feature or extension requires another to be defined first.
- `number` - feature version number, usually a string interpreted as *majorNumber:minorNumber*. Example: `4.2`.
- `comment` - arbitrary string (unused)

### 13.2 Contents of `<feature>` tags

Zero or more `<require>` and `<remove>` tags (see section 16), in arbitrary order. Each tag describes a set of interfaces that is respectively required for, or removed from, this feature, as described below.

### 13.3 Example of a <feature> tag

```
<feature api="gl" name="GL_VERSION_3_0" number="3.0">
  <require>
    <enum name="GL_COMPARE_REF_TO_TEXTURE" />
    <enum name="GL_CLIP_DISTANCE0" />
    <command name="glEndTransformFeedback" />
  </require>
  <require profile="compatibility">
    <enum name="GL_INDEX" />
  </require>
</feature>
```

When processed into a C header for the compatibility profile of OpenGL, this results in (assuming the usual definitions of these GL interfaces):

```
#ifndef GL_VERSION_3_0
#define GL_VERSION_3_0 1
#define GL_COMPARE_REF_TO_TEXTURE 0x884E
#define GL_CLIP_DISTANCE0 0x3000
#define GL_INDEX 0x8222
typedef void (APIENTRY PFNGLENDTRANSFORMFEEDBACKPROC) (void);
#ifdef GL_GLEXT_PROTOTYPES
GLAPI void APIENTRY glEndTransformFeedback (void);
#endif
#endif /* GL_VERSION_3_0 */
```

If processed into a header for the core profile, the definition of GL\_INDEX would not appear.

## 14 Extension Blocks (<extensions> tag)

The <extensions> tag contains definitions of each of the extensions which are defined for the API.

### 14.1 Attributes of <extensions> tags

None.

### 14.2 Contents of <extensions> tags

Each <extensions> block contains zero or more <extension> tags, each describing an API extension, in arbitrary order (although they are typically ordered by sorting on the extension name, to improve human readability).

## 15 API Extensions (<extension> tag)

API extensions are described in individual <extension> tags. An extension is the set of interfaces defined by a particular API extension specification, such as ARB\_multitexture. <extension> is similar to <feature>, but instead of having version and profile attributes, instead has a supported attribute, which describes the set of API names which the extension can potentially be implemented against.

### 15.1 Attributes of <extension> tags

- `supported` - a regular expression, with an implicit `^` and `$` bracketing it, which should match the `api` tag of a set of <feature> tags.
- `protect` - an additional preprocessor token used to protect an extension definition. Usually another feature or extension name. Rarely used, for odd circumstances where the definition of an extension requires another to be defined first.
- `comment` - arbitrary string (unused)

### 15.2 Contents of <extension> tags

Zero or more <require> and <remove> tags (see section 16), in arbitrary order. Each tag describes a set of interfaces that is respectively required for, or removed from, this extension, as described below.

### 15.3 Example of an <extensions> tag

```
<extensions>
  <extension name="GL_ARB_robustness" supported="gl|glcore" >
    <require>
      <enum name="GL_NO_ERROR" />
      <command name="glGetGraphicsResetStatusARB" />
    </require>
    <require api="gl" profile="compatibility">
      <command name="glGetnMapdvARB" />
    </require>
  </extension>
</extensions>
```

The supported attribute says that the extension can be supported for either the GL compatibility (gl) or GL core (glcore) API profiles, but not for other APIs. When processed into a C header for the core profile of OpenGL, this results in (assuming the usual definitions of these GL interfaces):





- `name` - name of the enumerant (required).
  - `comment` - arbitrary string (optional and unused).
- `<type>` specifies a required (or removed) type defined in a `<types>` block. Most types are picked up implicitly by using the `<p_type>` tags of commands, but in a few cases, additional types need to be specified explicitly (it is unlikely that a type would ever be removed, although this usage is allowed by the schema). The tag has no content, but contains elements:
    - `name` - name of the type (required).
    - `comment` - arbitrary string (optional and unused).

## 17 General Discussion

### 17.1 Stability of the XML Database and Schema

The new registry schema, scripts, and databases are evolving in response to feedback and to Khronos' own wishlist. This means the XML schema is subject to change, although most such change will probably be confined to adding attributes to existing tags. The XML databases such as `gl.xml` will evolve in response to schema changes, to new extensions and API versions, and to general cleanup, such as canonicalization of the XML or sorting of `<command>` and `<extension>` tags by name. Changes to the schema will be described in the change log of this document (see section 18). Changes to the `.xml` files will be described in Github revision history.

### 17.2 Feature Enhancements to the Registry

There are lots of tools and additional tags that would make the XML format more expressive and the tools more capable. Khronos is open to hosting additional processing scripts for other purposes related to the definition of the API and bindings to it.

A partial wishlist follows:

- Enhance `<command>` `<alias>` tags to describe more relaxed sorts of aliases, such as commands equivalent in behavior and interface, but using different GLX protocol (this might be called a *client-side alias* or something of the sort).

### 17.3 Type Annotations and Relationship to `.spec` Files

The initial releases of the XML Registry did not include type annotation and array length information from the old `.spec` files, which generated a number of complaints from people writing bindings of OpenGL to languages other than C. The majority of these annotations have now been added to the XML, in the form of `<group>` tags (see section 7) defining groups of related enumerants, `group` attributes on `<proto>` and `<param>` tags specifying that the corresponding return type belongs to a group, and `len` attributes on `<param>` tags specifying the array length of the corresponding parameter.

There are many caveats regarding these annotations. For the most part they date from SGI's OpenGL 1.1 implementation, and have not been updated. The defined `<group>`s therefore do not cover many API parameters which **could** be given a group, and in many cases, the defined `<group>`s have not been updated to add new enumerants from later versions of OpenGL and OpenGL extensions. The group names are often somewhat misleading (with imbedded vendor extension tags, while the corresponding features have been promoted).

The `<group>`s added to `gl.xml` are the enumerant groups defined in `enum.spec`, and `enumext.spec`. Many additional group names were used in the annotations in `gl.spec`, and they fall in several categories discussed below.

### 17.3.1 Simple API Type Aliases

Group names that were simply an alias for a GL type have been left entirely out of `gl.xml`, since they offer no useful additional information.

For example, a parameter described as `UInt32` in `gl.spec` is not annotated with `group="UInt32"` in `gl.xml`, because this offers no information about the parameter not already present in its declaration as `<ptype>GLuint</ptype>`.

A few examples of such groups are `cl_context`, `handleARB`, `Int32`, and `sync`.

### 17.3.2 Numeric Constraints

Group names representing some type of numerical constraint on a value are retained in `<proto>` and `<param>` group attributes, but no definition exists. This is because the existing `<group>` mechanism can only describe constraints to specific enumerant names.

This is not a regression relative to the `.spec` files, which also did not describe the meaning of these groups.

A few examples of such groups are `CheckedFloat32`, `ClampedFixed`, and `CoordD`.

### 17.3.3 GL Object Names

Group names representing an object name obtained from e.g. a `glGen*` command, such as a display list name, are retained in `<proto>` and `<param>` group attributes, but no definition exists. This is because the existing `<group>` mechanism can only describe constraints to specific enumerant names.

This is not a regression relative to the `.spec` files, which also did not describe the meaning of these groups.

A few examples of such groups are `List` and `Path`.

### 17.3.4 Groups Not Defined Yet

Group names representing enumerant groups which were not defined in `enum.spec` and `enumext.spec` are retained in `<proto>` and `<param>` group attributes, but

no definition exists. Such groups usually are a result of vendors contributing `gl.spec` entries for an extension without contributing corresponding `enum.spec` entries.

This is not a regression relative to the `.spec` files, which also did not describe the meaning of these groups.

A few examples of such groups are `ArrayObjectPNameATI`, `BinormalPointerTypeEXT`, and `SpriteParameterNameSGIX`.

### 17.3.5 Other Groups

There are probably a few groups which are present in `gl.xml` but should not be. These can be gradually cleaned up over time.

### 17.3.6 Validating Groups

The `genheaders.py` script has been enhanced to allow validating groups during header generation. Invoking it with the `-validate` option, for example via:

```
$ genheaders.py -validate -registry gl.xml
```

will generate a text dump on standard output showing all parameter type group attributes which do **not** have corresponding `<group>` tags.

## 18 Change Log

- 2020-10-27 - Add `class` attribute to `<param>` tags (github pull request 428), update document title and make minor tweaks to reflect this not being the only Khronos XML API schema in use, and having migrated from Subversion to Github.
- 2020-01-11 - Add `group` attribute to `<enum>` tags, and deprecate `<group>` tags (github issue 335).
- 2017-01-20 - Update for the move to the new github repository.
- 2014-06-16 - Add `<vendor>` attribute to `<unused>` tags.
- 2013-09-17 - Add `<comment>` attribute to `<command>` tags within `<commands>`.
- 2013-06-24 - Add `<groups>` and `<group>` tags, renamed `<enums>` attribute `class` to `group`, and add parameter type annotation attributes `group` and `len` to both `<proto>` and `<param>`. Add section 17.3 discussing limitations of these annotations. Still need to add examples of the annotation attributes.
- 2013-06-19 - Added `<extensions>` tag block as a wrapper around groups of `<extension>` tags, to ease XML transformations such as sorting extensions by name.
- 2013-06-14 - Expanded description of tags, better formatting
- 2013-06-12 - First release, text file format

## Index

`ialias`<sub>i</sub>, 11, 17  
`apientry`<sub>i</sub>, 6  
`command`<sub>i</sub>, 8, 10, 11, 16, 17, 19  
`commands`<sub>i</sub>, 4, 6, 10, 13, 16, 19  
`comment`<sub>i</sub>, 5, 19  
`enum`<sub>i</sub>, 8–10, 16, 19  
`enums`<sub>i</sub>, 4, 6, 8–10, 16, 19  
`extension`<sub>i</sub>, 4, 14–17, 19  
`extensions`<sub>i</sub>, 6, 14, 15, 19  
`feature`<sub>i</sub>, 4, 6, 13–16  
`glx`<sub>i</sub>, 11  
`group`<sub>i</sub>, 7–9, 17–19  
`groups`<sub>i</sub>, 5, 7, 19  
`name`<sub>i</sub>, 6, 12  
`param`<sub>i</sub>, 8, 11, 12, 17–19  
`proto`<sub>i</sub>, 8, 11, 12, 17–19  
`ptype`<sub>i</sub>, 12, 17  
`registry`<sub>i</sub>, 4, 5  
`remove`<sub>i</sub>, 13, 15, 16  
`require`<sub>i</sub>, 13, 15, 16  
`type`<sub>i</sub>, 6, 12, 17  
`types`<sub>i</sub>, 4–6, 17  
`unused`<sub>i</sub>, 8, 10, 19  
`vecequiv`<sub>i</sub>, 11  
`vendor`<sub>i</sub>, 19

alias, 9  
api, 5, 6, 9, 13, 15, 16

class, 12, 19  
comment, 6, 8, 10, 11, 13, 15–17

end, 8, 10  
enums, 10

group, 8, 9, 11, 12, 17–19

len, 12, 17, 19

name, 6–9, 11, 13, 15–17  
namespace, 8, 10  
number, 13

profile, 15, 16

protect, 13, 15

requires, 6

start, 8, 10  
supported, 15

type, 8, 9

value, 9  
vendor, 8, 10  
version, 15